

VERİ TABANI İŞLEMLERİ

Veri Tabanı Nedir?

Veri tabanları (**database**), tuttukları veriler hakkında bilgi sahibi olan programlardır. Veri tabanları, yazdığımız uygulamadan bağımsız olarak verilerimizi sistematik olarak saklar, erişilebilir kılar ve yönetmemizi sağlar. Veri tabanı kullanmak bize veri tutarlılığı, veri paylaşımı, veri güvenliği ve veri analitiği için bir altyapı sağlar.

Veri tabanı;

- Kullanıcılardan gelen sorgu taleplerini analiz eder,
- Analiz sonrasında varsa mevcut veri üzerinde değişiklik yapar veya istenen veriyi belirler
- Sorgu sonucunu kullanıcının erişebileceği formda sunar.

Veri tabanında sorgular, **Structured Query Language-SQL** dili ile ilişkisel veri tabanında veya NoSQL diliyle ilişkisel olmayan veri tabanlarında yapılmaktadır.

Veri tabanı yönetim sistemleri (VTYS) aracılığıyla veri tabanları güvenli ve ölçeklenebilir şekilde yönetilebilmektedir. Veri tabanı yönetim sistemi verinin fiziksel veya mantıksal alanda bulunduğunu belirler ve gelen sorguları bu kapsamda ayırtırmaktadır. Sorguları hızlandırmak için indeksler oluşturulup veriye hızlıca erişilmesi sağlanmaktadır.

Bir uygulama geliştirmeye başladığınızda yapacağınız ilk tercihlerden biri SQL veya NoSQL Veritabanı kullanıp kullanmayacağınızdır. Verileri düzenlemek çok zor bir iştir ve düzenlediğimizde verilerimizi çeşitli türlere göre kategorize ederiz.

PostgreSQL, MySQL, Microsoft SQL Server, Oracle Database veya SQLite gibi geleneksel bir veri tabanı şema kullanır. SQL veri tabanları şema kullanır ve saklanan verileri düzenlemek daha kolaydır.

MongoDB, Redis, CouchDB, Cassandra, Elasticsearch, Couchbase ve LiteDB gibi NoSQL veri tabanları SQL veri tabanları kadar veri tutarlılığı sağlamaz ve verileri düzenlemek çok daha zordur. Daha çok **iz kayıtları (log)**, belge ve resim saklamak için kullanılırlar.

SQL ve NoSQL arasındaki temel fark, verilerin nasıl depolandığı ve nasıl dışarıya aktarıldığıdır. Bir veri tabanı projesini yazabilmeniz için ilk önce ilgili veri tabanına karar verip projenizin çalıştığı bilgisayara ya da projenizle aynı ağda ve erişilebilir olan bir bilgisayara kurmanız gerekir.

Erişilebilir bir veri tabanınız varsa o veri tabanına ilişkin bileşenleri projemize dahil ederek veri tabanına bağlanıp gerekli veri işlemlerini gerçekleştirebiliriz.

Bu tür bileşenleri projemize dahil etmek için *Tools → NuGet Package Manager → Package Manager Console* üzerinden komut vererek bileşenleri projemize dahil ederiz. Veri Tabanı Sunucuları için örnek;

```
Install-Package Microsoft.Data.Sqlite
Install-Package Microsoft.Data.SqlClient
Install-Package LiteDb
```

Bağlantı Dizgisi

Projemizde kullanacağımız veri tabanları uygulamamızdan ayrı bir programdır. Bu veri tabanlarına bağlanmamızı sağlayan metinleri **dizgi (string)** değişkenleri olarak tanımlarız. Bu değişkenlere **bağlantı dizgisi (connection string)** adı veririz. Bu değişken genellikle veri tabanı yönetim sistemine iletilen kullanıcı adı, şifre ve şema bilgileri içerir.

```
string sqliteConnectionString="Data Source=hello.db";
var sqliteConnection = new SqliteConnection(sqliteConnectionString);

string nosqlConnectionString=@"MyData.db";
var nosqlConnection = new LiteDatabase(nosqlConnectionString);
```

Küçük ve yerel bilgisayarınızda çalışabilecek Veri tabanlarının bağlantı dizgileri kısa ve öz olur. Ancak kurumsal ve ölçeklenebilir Veri tabanlarının bağlantı dizgisi daha karmaşık olabilir. Bu gibi bağlantı

dizgilerini oluşturmak için `DbConnectionStringBuilder`, `SqlConnectionStringBuilder` gibi yardımcı sınıflar kullanılır;

```
DbConnectionStringBuilder builder1 = new DbConnectionStringBuilder();
builder1.ConnectionString = @"Data Source=D:\Data\MyDb.mdb"; //Access DataBase
builder1.Add("Provider", "Microsoft.Jet.Oledb.4.0");
builder1.Add("Jet OLEDB:Database Locking Mode", 1);
OleDbConnection oledbConnect = new OleDbConnection(builder1.ConnectionString);
var builder2 = new SqlConnectionStringBuilder
{
    DataSource = "(local)",
    UserID = "sa",
    Password = "123456*",
    InitialCatalog = "CSharpDB"
};
var SqlConnection = new SqlConnection(builder2.ConnectionString);
var SqlConnection2 = new SqlConnection("server=(local);initial catalog=CSharpDB");
```

Veri kaynakları genellikle hem mali hem de tükettikleri kaynak açısından pahalı sistemlerdir. Pahalı olan bu kaynaklar başka yazılımlar ile ortak kullanılırlar. Bu nedenle veri kaynaklarına olan bağlantılarımızı sınırlı tutmalıyız. Bu tür kaynakları etkin kullanmak için `using` saklı kelimesi kullanılır. `using` kelimesi veri kaynağı gibi değerli kaynaklara ilişkin işlem yapılırken bir hata meydana geldiğinde, bağlantıya ilişkin tüm nesneleri `Dispose` yöntemi ile bellekten siler yani `Dispose` yönteminin çağırılmasını garanti eder.

İlişkisel Veri Tabanı İşlemleri

Veri tabanını kullanabilmek için ilk önce bir `bağlantı` (`connection`) kurmak gerekir. Sonrasında veri tabanı üzerinde işlem yapacak `SQL komutu` (`command`) nesnesi hazırlanır ve hazırlanan komut `icra` (`execute`) edilir.

Veri Tabanında Tablolar Oluşturma

```
// See https://aka.ms/new-console-template for more information
using Microsoft.Data.Sqlite;
var connectionString = @"Data Source=D:\CSharp.sqlite";
using (var connection = new SqliteConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText =
        @"
        DROP TABLE IF EXISTS Ogrenci;
        CREATE TABLE Ogrenci (
            OgrenciId INTEGER PRIMARY KEY AUTOINCREMENT,
            Adi text NOT NULL,
            Soyadi text NOT NULL
        );
        DROP TABLE IF EXISTS Ders;
        CREATE TABLE Ders(
            DersId INTEGER PRIMARY KEY AUTOINCREMENT,
            DersAdi NVARCHAR(50)
        );
        ";
    /*
        Yukarıfaki gibi Noktalı virgül karakteriyle ayırarak
        birden fazla sorgu çalıştırabiliriz.
    */
    int rowsAffected = command.ExecuteNonQuery(); //Veri getirmeyecek bir sorgu!
    Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
    command.CommandText =
        @"
```

```

INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ali', 'Veli');
INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ayşe', 'Fatma');
INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Mehmet', 'Can');
INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Zeynep', 'Yılmaz');
INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ahmet', 'Demir');
INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Fatma', 'Kaya');
INSERT INTO Ders (DersAdi) VALUES ('Matematik');
INSERT INTO Ders (DersAdi) VALUES ('Fizik');

";
rowsAffected = command.ExecuteNonQuery();
Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
connection.Close();
}

```

SQLite/SQL Compact Toolbox eklentisini Visual Studio'ya kurarak oluşturduğumuz SQLite veri tabanını açıp sorgulayabiliriz²⁹.

Sorgu Parametreleri

Veri tabanı için yazılacak sorgulara parametre metin olarak yazılma yerine bir değişkene bağlanabilir;

```

// See https://aka.ms/new-console-template for more information
using Microsoft.Data.Sqlite;
var connectionString = @"Data Source=D:\CSharp.sqlite";
using (var connection = new SqliteConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText =
@"
        INSERT INTO Ogrenci (Adi, Soyadi) VALUES ($Ad, $Soyad);
";
    string ad="İlhan", soyad="Özkan";
    command.Parameters.AddWithValue("$Ad", ad);
    command.Parameters.AddWithValue("$Soyad", soyad);
    int rowsAffected= command.ExecuteNonQuery();
    Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
    connection.Close();
}
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");

```

Veri Tabanından Veri Okuma

Veri tabanına eklenmiş kayıtları da sorgulatıp öğrenebiliriz;

```

// See https://aka.ms/new-console-template for more information
using Microsoft.Data.Sqlite;
var connectionString = @"Data Source=D:\CSharp.sqlite";
using (var connection = new SqliteConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText = @"SELECT OgrenciId, Adi, Soyadi FROM Ogrenci;";
    using (SqliteDataReader reader = command.ExecuteReader()) //Veri getirecek bir sorgu!
    {
        while (reader.Read())
        {
            var öğrenciId = reader.GetString(0);
            var adı = reader.GetString(1);
            var soyadı = reader.GetString(2);
            Console.WriteLine($"{öğrenciId} {adı} {soyadı}");
        }
    }
}

```

²⁹ <https://marketplace.visualstudio.com/items?itemName=ErikEJ.SQLServerCompactSQLiteToolbox>

```

    }
}
connection.Close();
}

```

İşlemler

İşlemler (**transaction**), birden fazla SQL ifadesini tek bir iş biriminde gruplandırmanıza ve bu iş biriminin veri tabanına tek bir atomik birim olarak kaydedilmesine olanak tanır. İş birimindeki herhangi bir ifade başarısız olursa, önceki ifadeler tarafından yapılan değişiklikler geri alınabilir. İşlem başlatıldığında veri tabanının ilk durumu korunur. Veri tabanında aynı anda çok sayıda değişiklik yapıldığı durumlarda faydalıdır.

Kurumsal veri tabanlarının aksine SQLite veri tabanında, aynı anda yalnızca bir işlemin beklemede olan değişikliklere sahip olmasına izin verilir. Bu durum **eş zamanlılık** (**concurrency**) olarak adlandırılır. Bu nedenle, **SQLiteCommand** ile başlanan işte **BeginTransaction** ve **Execute** yöntemlerine yapılan çağrılar, başka bir işlemin tamamlanması çok uzun sürerse zaman aşımına uğrayabilir³⁰.

```

// See https://aka.ms/new-console-template for more information
using Microsoft.Data.Sqlite;
var connectionString = @"Data Source=D:\CSharp.sqlite";
using (var connection = new SqliteConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText =
        @"
        DROP TABLE IF EXISTS OgrencilerinAldigiDersler;
        CREATE TABLE Burslar (
            OgrenciId INTEGER NOT NULL,
            BursMiktari REAL NOT NULL
        );
        INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (1,1000.0);
        INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (2,2000.0);
        INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (3,3000.0);
        INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (4,4000.0);
        INSERT INTO Burslar (OgrenciId, BursMiktari) VALUES (5,3000.0);
        ";
    int rowsAffected = command.ExecuteNonQuery();
    Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
    using (var transaction = connection.BeginTransaction(deferred: true))
    {
        var readCommand = connection.CreateCommand();
        long ogrenciId = 1;
        readCommand.CommandText = @"SELECT BursMiktari FROM Burslar Where (OgrenciId=@Id)";
        readCommand.Parameters.AddWithValue("@Id", ogrenciId);
        var value = (double) readCommand.ExecuteScalar();
        var writeCommand = connection.CreateCommand();
        writeCommand.CommandText =
            @"UPDATE Burslar SET BursMiktari = $newValue Where OgrenciId = @Id";
        writeCommand.Parameters.AddWithValue("$newValue", value + 6000.0);
        writeCommand.Parameters.AddWithValue("@Id", ogrenciId);
        writeCommand.ExecuteNonQuery();
        transaction.Commit();
    }
    connection.Close();
}

```

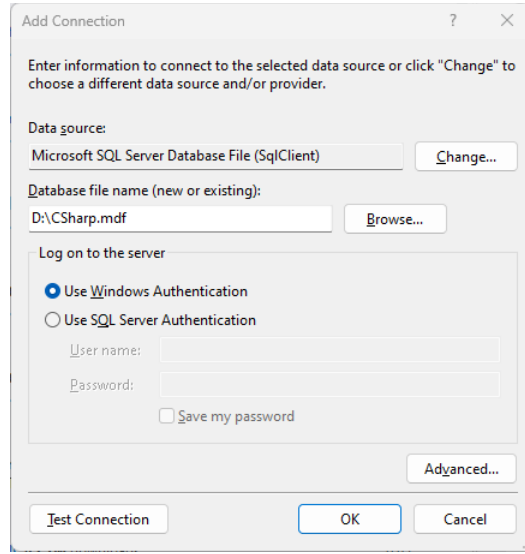
³⁰ <https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/transactions>

Veri Kaynaklarına Erişim Yöntemleri

.Net Framework ile veri kaynaklarına iki stratejiyle erişim yapılır;

1. **Bağlı (connected)**: Bu yöntemde veri üzerindeki değişiklikler doğrudan veri kaynağı üzerinde yapılır. Veri kaynakları pahalı yatırımlarla kurulduğundan, kaynağı en az meşgul edecek işlemler için veya zorunlu durumlarda kullanılır. .NET teknolojisinde bunu yukarıdaki örneklerde verildiği gibi, **DataReader** nesnesi ile yaparız.
2. **Kopuk (disconnected)**: Bu yöntemde veriler bellekte geçici bir yere aktarılırlar. **Veri seti (DataSet)** olarak adlandırılan bu yer, sanki bir veri kaynağıymış gibi kullanılır. Veri üzerinden işlemler tamamlandığında, bellekteki değişiklikler veri kaynağına kaydedilir. Veri setleri, hafızada oluşturulan bir veri tabanlarıdır.

Veri Seti için *Microsoft SQL Server LocalDB* sürümünü kullanacağız. Bunun için *Tools → Connect To Database...* seçilir. Açılan pencerede yeni bir veri tabanı oluşturulur;



Şekil 34. MSSQL Veritabanı Oluşturma

Oluşan veri tabanına *View → Server Explorer* menüsünden ulaşılabilir. Eğer .NET Core CLI ile proje oluşturmuş iseniz veri tabanını yönetmek için *SQL Server Management Studio* yazılımını kullanabilirsiniz³¹.

```
// See https://aka.ms/new-console-template for more information
using Microsoft.Data.SqlClient;
using System.Data;
var connectionString = @"Data Source=(LocalDB)\MSSQLLocalDB;"
                        + @"AttachDbFilename=D:\CSharp.mdf;"
                        + @"Integrated Security=True;Connect Timeout=30";
using (var connection = new SqlConnection(connectionString))
{
    connection.Open();
    var command = connection.CreateCommand();
    command.CommandText =
@"
DROP TABLE IF EXISTS Ogrenci;
CREATE TABLE Ogrenci (
    OgrenciId INT PRIMARY KEY IDENTITY (1, 1),
    Adi NVARCHAR (50) NOT NULL,
    Soyadi NVARCHAR (50) NOT NULL
);
DROP TABLE IF EXISTS Ders;
CREATE TABLE Ders(
    DersId INT PRIMARY KEY IDENTITY (1, 1),

```

³¹ <https://learn.microsoft.com/en-us/ssms/install/install>

```

        DersAdi NVARCHAR(50) NOT NULL
    );
";
int rowsAffected = command.ExecuteNonQuery();
Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
command.CommandText =
@"
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ali', 'Veli');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ayşe', 'Fatma');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Mehmet', 'Can');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Zeynep', 'Yılmaz');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Ahmet', 'Demir');
    INSERT INTO Ogrenci (Adi, Soyadi) VALUES ('Fatma', 'Kaya');
    INSERT INTO Ders (DersAdi) VALUES ('Matematik');
    INSERT INTO Ders (DersAdi) VALUES ('Fizik');
";
rowsAffected = command.ExecuteNonQuery();
Console.WriteLine($"{rowsAffected} kadar kayıt etkilendi.");
DataSet dataSet = new DataSet();
dataSet.Tables.Add("Ogrenci");
dataSet.Tables.Add("Ders");
SqlDataAdapter dataAdapterOgrenci = new SqlDataAdapter("Select * From Ogrenci",
    connection);

dataAdapterOgrenci.Fill(dataSet.Tables[0]);
SqlDataAdapter dataAdapterDers = new SqlDataAdapter("Select * From Ders",
    connection);

dataAdapterDers.Fill(dataSet.Tables[1]);
/* veriler veritabanından okunup verisetine yüklendi. veritabanına bağlatı
   (connection) sadece bu an için yapıldı. */
connection.Close();
Console.WriteLine("OgrenciId,Adı,Soyadı");
foreach (DataRow row in dataSet.Tables[0].Rows)
{
    Console.WriteLine("{0},{1},{2}",row[0], row[1], row[2]);
}
Console.WriteLine("DersId,DersAdi");
foreach (DataRow row2 in dataSet.Tables[1].Rows)
{
    Console.WriteLine("{0},{1}",row2[0], row2[1]);
}
}

```

NoSQL Veri Tabanı

NoSQL veri tabanları ilişkisel veri tabanları kadar veri tutarlılığı sağlamaz ve verileri düzenlemek çok daha zordur. Bu nedenle genellikle veri yazmak ve okumak için kullanılır. Aşağıda bir okuldaki öğrenciler ve diplomalarının saklandığı bir NoSQL örneği verilmiştir;

```

// See https://aka.ms/new-console-template for more information
using LiteDB;
using (var db = new LiteDatabase(@"D:\CSharp.litedb"))
{
    var öğrenciler = db.GetCollection<Ogrenci>("Ogrenciler");
    var öğrenci = new Ogrenci
    {
        Adı = "İlhan Özkan",
        EPosta = "ilhanozkan@outlook.com",
        CepTelefonu = "03121234567"
    };
    // Yeni Öğrenci koleksiyona eklenir (Id otomatik artar)
    öğrenciler.Insert(öğrenci);
    var result = öğrenciler.Find(x => x.EPosta == "ilhanozkan@outlook.com").FirstOrDefault();
}

```

```

        Console.WriteLine($"{result.Id}-{result.Adı}");
        öğrenci.CepTelefonu = "04621234567";
        öğrenciler.Update(öğrenci);
        var diplomalar = db.GetCollection<Diploma>("Diplomalar");
        byte[] fileBytes = File.ReadAllBytes(@"D:\lisans.pdf"); //PDF dosyayı veritabanına
        var diploma = new Diploma
        {
            ÖğrenciId = öğrenci.Id,
            PdfDocument = fileBytes
        };
        diplomalar.Insert(diploma);
    }

    public class Diploma
    {
        public int Id { get; set; }
        public int ÖğrenciId { get; set; }
        public byte[] PdfDocument { get; set; }
    }

    public class Öğrenci
    {
        public int Id { get; set; }
        public string Adı { get; set; }
        public string EPosta { get; set; }
        public string CepTelefonu { get; set; }
    }
}

```

Veri tabanına yazdırılan nesneleri okumak için aşağıdaki kod kullanılabilir;

```

// See https://aka.ms/new-console-template for more information
using LiteDB;
using (var db = new LiteDatabase(@"D:\CSharp.Litedb"))
{
    var öğrenciler = db.GetCollection<Öğrenci>("Öğrenciler");
    var liste = öğrenciler.FindAll();
    foreach (var o in liste)
        Console.WriteLine($"{o.Id}-{o.Adı}-{o.EPosta}-{o.CepTelefonu}");
    var diplomalar = db.GetCollection<Diploma>("Diplomalar");
    var result = diplomalar.Find(x => x.ÖğrenciId == 1).FirstOrDefault();
    File.WriteAllBytes(@"D:\lisans.new.pdf", result.PdfDocument);
    //veritabanından dosya sistemine dosyayı yazdık.
}

public class Diploma
{
    public int Id { get; set; }
    public int ÖğrenciId { get; set; }
    public byte[] PdfDocument { get; set; }
}

public class Öğrenci
{
    public int Id { get; set; }
    public string? Adı { get; set; }
    public string? EPosta { get; set; }
    public string? CepTelefonu { get; set; }
}

```